

Question No:2

DS

Session

Queue

Question Information

Level 1

Challenge 52

Question description

There is a bit string consisting of n bits. Then, there are some changes that invert one given bit. Your task is to report, after each change, the length of the longest substring whose each bit is the same.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq x_i \leq n$

Input Format

The first input line has a bit string consisting of n bits. The bits are numbered $1, 2, \dots, n$.

The next line contains an integer m : the number of changes.

The last line contains m integers $x_1, x_2, \dots, x_m$ describing the changes.

Output Format

After each change, print the length of the longest substring whose each bit is the same.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

001011

3

3 2 5

EXPECTED OUTPUT

4 2 3

Test Case 2

INPUT (STDIN)

101010

4

1 3 2 6

EXPECTED OUTPUT

2 4 2 2

Code:

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 200005
5
6  char s[MAX];
7
8  struct Node
9  {
10     int leftBit;
11     int rightBit;
12     int prefix;
13     int suffix;
14     int best;
15     int len;
16 };
17
18 struct Node tree[4 * MAX];
19
20 struct Node merge(struct Node a, struct Node b)
21 {
22     struct Node res;
23
24     res.len = a.len + b.len;
25
26     res.leftBit = a.leftBit;
27     res.rightBit = b.rightBit;
28
29     /* Prefix */
30     res.prefix = a.prefix;
31
32     if (a.prefix == a.len && a.rightBit == b.leftBit)
33     {
34         res.prefix = a.len + b.prefix;
35     }
36
37     /* Suffix */
```

```

38     res.suffix = b.suffix;
39
40     if (b.suffix == b.len && a.rightBit == b.leftBit)
41     {
42         res.suffix = b.len + a.suffix;
43     }
44
45     /* Best */
46     res.best = a.best;
47
48     if (b.best > res.best)
49     {
50         res.best = b.best;
51     }
52
53     if (a.rightBit == b.leftBit)
54     {
55         int middle = a.suffix + b.prefix;
56
57         if (middle > res.best)
58         {
59             res.best = middle;
60         }
61     }
62
63     return res;
64 }
65
66 void build(int node, int start, int end)
67 {
68     if (start == end)
69     {
70         int bit = s[start] - '0';
71
72         tree[node].leftBit = bit;
73         tree[node].rightBit = bit;

```

```

74         tree[node].prefix = 1;
75         tree[node].suffix = 1;
76         tree[node].best = 1;
77         tree[node].len = 1;
78
79         return;
80     }
81
82     int mid = (start + end) / 2;
83
84     build(node * 2, start, mid);
85     build(node * 2 + 1, mid + 1, end);
86
87     tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
88 }
89
90 void update(int node, int start, int end, int pos)
91 {
92     if (start == end)
93     {
94         /* Flip the bit */
95         if (s[pos] == '0')
96             s[pos] = '1';
97         else
98             s[pos] = '0';
99
100         int bit = s[pos] - '0';
101
102         tree[node].leftBit = bit;
103         tree[node].rightBit = bit;
104         tree[node].prefix = 1;
105         tree[node].suffix = 1;
106         tree[node].best = 1;
107         tree[node].len = 1;
108
109         return;

```

```

110     }
111
112     int mid = (start + end) / 2;
113
114     if (pos <= mid)
115     {
116         update(node * 2, start, mid, pos);
117     }
118     else
119     {
120         update(node * 2 + 1, mid + 1, end, pos);
121     }
122
123     tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
124 }
125
126 int main()
127 {
128     int n, m;
129
130     scanf("%s", s);
131
132     n = strlen(s);
133
134     scanf("%d", &m);
135
136     build(1, 0, n - 1);
137
138     for (int i = 0; i < m; i++)
139     {
140         int x;
141
142         scanf("%d", &x);
143
144         /* Input positions are 1-based */
145         x--;

```

```
146
147         update(1, 0, n - 1, x);
148
149         printf("%d", tree[1].best);
150
151         if (i < m - 1)
152         {
153             printf(" ");
154         }
155     }
156
157     printf("\n");
158
159     return 0;
160
```

Output:

Test Case-1:

```
001011
```

```
3
```

```
3 2 5
```

```
4 2 3
```

```
-----
```

```
Process exited after 18.98 seconds with return value 0
```

```
Press any key to continue . . . |
```

Test Case-2:

```
101010
```

```
4
```

```
1 3 2 6
```

```
2 4 2 2
```

```
-----
```

```
Process exited after 17.14 seconds with return value 0
```

```
Press any key to continue . . . |
```

Keshav Kumar Rauniyar

CH.SC.U4CSE25236